Programming Language Principles, Design, and Implementation

Lecture 11 - Existential Types

Sean Moss

University of Birmingham

Where are we?

- **Part 1: Principles of programming**
  - Functional languages
  - Operational semantics
  - Type systems
  - **Abstract datatypes**
- Part 2: Compilers

Today

Existential types

- What are existential types?
- Abstract data types as existential types
- Existential types in System F

Further reading:

- "Types and Programming Languages", Pierce, Chapter 24

Recap: Abstraction

The idea:

- **Modularity**: a good approach to problem solving is to divide the problem into smaller independent problems
- implement solutions as collections of **independent modules**
- the user of a module should not have to understand the implementation details of the module to use it
- a module should be allowed to change without affecting its users
- **Abstraction**: hide implementation details
- only provide abstract interfaces to modules

Abstract data types (ADTs) allow abstracting over functions and data

They enforce an **abstraction barrier**

They can be implemented using **Modules** in Haskell

Recap: Data Abstraction Using Modules in Haskell

An abstract Counter data type:

```
1  module Counter(Counter,new,get,inc) where
2  data Counter = C Integer
3
4  new :: Counter
5  new = 0
6
7  get :: Counter -> Integer
8  get (C c) = c
9
10 inc :: Counter -> Counter
11 inc (C c) = C (c + 1)
```

- the module exports the abstract type Counter
- the interface: new, get, and inc
- and hides the type constructor C

What features do type systems need to have to support such types?

Existential Types—Intuition

We will now see how to capture data abstraction using **existential types** of the form $\exists\alpha.T$, which we will add to System F.

We saw that **universal types** of the form $\forall\alpha.T$ are used in System F to specify polymorphic functions.

- universal types: $\forall\alpha.T$
- type abstractions: $\lambda\alpha.M$
- type applications: $M\{T\}$

For example: in System F, the identity function is $\lambda\alpha.\lambda x:\alpha.x$

Universal vs. Existential:

- An element of type $\forall\alpha.T$
  - is a **function** $\lambda\alpha.M$
  - such that for all types U, $M[\alpha\backslash U]$ has type $T[\alpha\backslash U]$
- An element of type $\exists\alpha.T$
  - can be seen a **pair** $\langle U, M\rangle$
  - of a type U and an element M of type $T[\alpha\backslash U]$

Existential Types—Intuition

We will see later that members of existential types are slightly more complicated than pairs of the form $\langle U, M \rangle$.

Intuition: a pair $\langle U, M \rangle$ of a type U and an element M of type $T[\alpha \backslash U]$ can be seen as a **package** with

- a hidden type component—the **hidden representation (or witness) type**
- and a visible term component

If $\lambda x.M$ is an '**anonymous function**', a package $\langle U, M \rangle$ is a (very simplified) '**anonymous module**'.

Example:

- the package $\langle \mathbb{N}, \lambda x : \mathbb{N}.\mathtt{succ}\ x \rangle$
- has type $\exists \alpha.\alpha \to \alpha$

Can you find another example of a package that has that type?

Example:

- the package $\langle \mathbb{B}, \lambda x : \mathbb{B}.\mathtt{if}\ x\ \mathtt{then}\ \mathtt{false}\ \mathtt{else}\ \mathtt{true} \rangle$
- also has type $\exists \alpha.\alpha \to \alpha$

Existential Types—Intuition

We just saw that:

- the package $\langle \mathbb{N}, \lambda x : \mathbb{N}.\mathtt{succ}\ x \rangle$
- has type $\exists \alpha.\alpha \to \alpha$

Is that the only type that this package has?

- the package $\langle \mathbb{N}, \lambda x : \mathbb{N}.\mathtt{succ}\ x \rangle$
- also has type $\exists \alpha.\alpha \to \mathbb{N}$ because $(\alpha \to \mathbb{N})[\alpha \backslash \mathbb{N}] = \mathbb{N} \to \mathbb{N}$
- and type $\exists \alpha.\mathbb{N} \to \alpha$ because $(\mathbb{N} \to \alpha)[\alpha \backslash \mathbb{N}] = \mathbb{N} \to \mathbb{N}$
- and type $\exists \alpha.\mathbb{N} \to \mathbb{N}$ because $(\mathbb{N} \to \mathbb{N})[\alpha \backslash \mathbb{N}] = \mathbb{N} \to \mathbb{N}$

To help infer packages' types, we will annotate them as follows:

$$\mathtt{pack}\ \langle \mathbb{N}, \lambda x : \mathbb{N}.\mathtt{succ}\ x \rangle\ \mathtt{as}\ \exists \alpha.\alpha \to \alpha$$

The annotation provides the type interface of the package

Existential Types—Intuition

We just saw that:

- the package $\texttt{pack}\ \langle \mathbb{N}, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle\ \texttt{as}\ \exists\alpha.\alpha \to \alpha$
- has type $\exists\alpha.\alpha \to \alpha$

How can we use packages? We need to be able to unpack them!

We also introduce **unpacking** expressions of the form:

$$\texttt{unpack}\ \alpha, x\ =\ M\ \texttt{in}\ N$$

which

- extracts the witness part of the pack M and binds it to α
- extracts the component part of the pack M and binds it to x
- evaluate N, which can make use of α and x

Existential Types—Intuition

Can we handle our Counter example now?

A pack of the form $\texttt{pack}\ \langle T, M\rangle\ \texttt{as}\ U$ does not provide a convenient way of defining multiple functions within M

For this, we simply add "regular" pairs (record types would be more convenient):

- a pair is of the form $\langle M, N\rangle$
- if M has type T and N has type U then $\langle M, N\rangle$ has type $T \times U$ (left associative)
- we also introduce $\texttt{fst}\ M$ and $\texttt{snd}\ M$ operators to extract the first and second components of a pair, respectively

Our Counter example can now be defined as follows:

$$\begin{array}{rl} \texttt{pack} & \langle \mathbb{N}, \langle\langle \texttt{zero}, \lambda x : \mathbb{N}.x\rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x\rangle\rangle \\ \texttt{as} & \exists\alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha) \end{array}$$

Existential Types

We extend System F as follows:

Syntax:

$$
\begin{array}{rcl}
T & ::= & \alpha \mid \mathbb{B} \mid \mathbb{N} \mid T \to T \mid \forall\alpha.T \mid T \times T \mid \exists\alpha.T \\
M & ::= & x \mid \lambda x : T.M \mid M\ M \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ M\ \texttt{then}\ M\ \texttt{else}\ M \\
& \mid & \texttt{let}\ x\ =\ M\ \texttt{in}\ M \mid \texttt{zero} \mid \texttt{succ}\ M \mid \texttt{pred}\ M \mid \texttt{iszero}\ M \\
& \mid & \lambda\alpha.M \mid M\{T\} \\
& \mid & \langle M, M\rangle \mid \texttt{fst}\ M \mid \texttt{snd}\ M \\
& \mid & \texttt{pack}\ \langle T, M\rangle\ \texttt{as}\ T \mid \texttt{unpack}\ \alpha, x\ =\ M\ \texttt{in}\ M
\end{array}
$$

Values:

$$
\begin{array}{rcl}
V & ::= & \lambda x : T.M \mid \texttt{true} \mid \texttt{false} \mid \texttt{zero} \mid \texttt{succ}\ V \mid \lambda\alpha.M \\
& \mid & \langle V, V\rangle \mid \texttt{pack}\ \langle T, V\rangle\ \texttt{as}\ T
\end{array}
$$

Call-by-value evaluation contexts

$$
\begin{array}{rcl}
C & ::= & \bullet \mid C\ M \mid V\ C \mid \texttt{if}\ C\ \texttt{then}\ M\ \texttt{else}\ M \\
& \mid & \texttt{let}\ x\ =\ C\ \texttt{in}\ M \mid \texttt{pred}\ C \mid \texttt{iszero}\ C \mid \texttt{succ}\ C \\
& \mid & C\{T\} \mid \texttt{unpack}\ \alpha, x\ =\ C\ \texttt{in}\ M \mid \texttt{fst}\ C \mid \texttt{snd}\ C \\
& \mid & \langle C, M\rangle \mid \langle V, C\rangle \mid \texttt{pack}\ \langle T, C\rangle\ \texttt{as}\ T
\end{array}
$$

Existential Types

Call-by-value small-step operational semantics rules:

$$\frac{}{(\lambda x : T.M)\ V \longrightarrow_v M[x\backslash V]}\ \beta \qquad \frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]}\ \mathrm{CTX}_C$$

$$\frac{}{\texttt{if true then } M \texttt{ else } N \longrightarrow_v M}\ \mathrm{IteT} \qquad \frac{}{\texttt{if false then } M \texttt{ else } N \longrightarrow_v N}\ \mathrm{IteF}$$

$$\frac{}{\texttt{let } x = V \texttt{ in } M \longrightarrow_v M[x\backslash V]}\ \mathrm{LetV} \qquad \frac{}{(\lambda\alpha.M)\{T\} \longrightarrow_v M[\alpha\backslash T]}\ \mathrm{T}\beta$$

$$\frac{}{\texttt{pred zero} \longrightarrow_v \texttt{zero}}\ \mathrm{PredZ} \qquad \frac{}{\texttt{pred (succ } V\texttt{)} \longrightarrow_v V}\ \mathrm{PredS}$$

$$\frac{}{\texttt{iszero zero} \longrightarrow_v \texttt{true}}\ \mathrm{IsZZ} \qquad \frac{}{\texttt{iszero (succ } V\texttt{)} \longrightarrow_v \texttt{false}}\ \mathrm{IsZS}$$

$$\frac{}{\texttt{fst } \langle V, W\rangle \longrightarrow_v V}\ \mathrm{FstP} \qquad \frac{}{\texttt{snd } \langle V, W\rangle \longrightarrow_v W}\ \mathrm{SndP}$$

$$\frac{}{\texttt{unpack } \alpha, x = \texttt{pack } \langle T, V\rangle \texttt{ as } U \texttt{ in } N \longrightarrow_v N[\alpha\backslash T][x\backslash V]}\ \mathrm{UnP}$$

Existential Types

Let $P = \begin{array}{rl} \texttt{pack} & \langle \mathbb{N}, \langle \langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle \rangle \\ \texttt{as} & \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha) \end{array}$

What values do these expressions reduce to?

- $\texttt{unpack}\ \alpha, x = P\ \texttt{in}\ (\texttt{snd}\ x)\ (\texttt{fst}\ (\texttt{fst}\ x)) \longrightarrow_v^* \texttt{succ zero}$
- $\texttt{unpack}\ \alpha, x = P\ \texttt{in}\ (\texttt{snd}\ x)\ \texttt{zero} \longrightarrow_v^* \texttt{succ zero}$
- $\texttt{unpack}\ \alpha, x = P\ \texttt{in}\ (\texttt{snd}\ (\texttt{fst}\ x))\ (\texttt{fst}\ (\texttt{fst}\ x)) \longrightarrow_v^* \texttt{zero}$
- $\texttt{unpack}\ \alpha, x = P\ \texttt{in}\ (\texttt{snd}\ (\texttt{fst}\ x))\ ((\texttt{snd}\ x)\ (\texttt{fst}\ (\texttt{fst}\ x))) \longrightarrow_v^* \texttt{succ zero}$

Existential Types

Typing rules:

$$\frac{}{\Gamma, x : T \vdash x : T}\ \textsc{var} \qquad \frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x : T.M : T \to U}\ \textsc{abs} \qquad \frac{\Gamma \vdash M : T \to U \quad \Gamma \vdash N : T}{\Gamma \vdash M\ N : U}\ \textsc{app}$$

$$\frac{}{\Gamma \vdash \texttt{true} : \mathbb{B}}\ \textsc{t} \qquad \frac{}{\Gamma \vdash \texttt{false} : \mathbb{B}}\ \textsc{f} \qquad \frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \texttt{if}\ M\ \texttt{then}\ N\ \texttt{else}\ P : T}\ \textsc{ite}$$

$$\frac{}{\Gamma \vdash \texttt{zero} : \mathbb{N}}\ \textsc{z} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \texttt{succ}\ M : \mathbb{N}}\ \textsc{s} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \texttt{pred}\ M : \mathbb{N}}\ \textsc{s} \qquad \frac{\Gamma \vdash M : \mathbb{N}}{\Gamma \vdash \texttt{iszero}\ M : \mathbb{B}}\ \textsc{i}$$

$$\frac{\Gamma \vdash M : T \quad \Gamma, x : T \vdash N : U}{\Gamma \vdash \texttt{let}\ x\ =\ M\ \texttt{in}\ N : U}\ \textsc{let} \qquad \frac{\Gamma \vdash M : T}{\Gamma \vdash \lambda\alpha.M : \forall\alpha.T}\ \textsc{tabs} \qquad \frac{\Gamma \vdash M : \forall\alpha.T}{\Gamma \vdash M\{U\} : T[\alpha\backslash U]}\ \textsc{tapp}$$

$$\frac{\Gamma \vdash M : T \quad \Gamma \vdash N : U}{\Gamma \vdash \langle M, N\rangle : T \times U}\ \textsc{Pair} \qquad \frac{\Gamma \vdash M : T \times U}{\Gamma \vdash \texttt{fst}\ M : T}\ \textsc{Fst} \qquad \frac{\Gamma \vdash M : T \times U}{\Gamma \vdash \texttt{snd}\ M : U}\ \textsc{Snd}$$

$$\frac{\Gamma \vdash M : U[\alpha\backslash T]}{\Gamma \vdash \texttt{pack}\ \langle T, M\rangle\ \texttt{as}\ \exists\alpha.U : \exists\alpha.U}\ \textsc{Pack} \qquad \frac{\Gamma \vdash M : \exists\alpha.T \quad \Gamma, x : T \vdash N : U}{\Gamma \vdash \texttt{unpack}\ \alpha, x\ =\ M\ \texttt{in}\ N : U}\ \textsc{Unpack}$$

In addition TABS requires that $\alpha \notin \mathrm{FV}(\Gamma)$
and Unpack requires that $\alpha \notin \mathrm{FV}(U)$

Existential Types

Let $P = \begin{array}{rl} \texttt{pack} & \langle \mathbb{N}, \langle \langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle \rangle \\ \texttt{as} & \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha) \end{array}$

Provide a derivation that P has type $\exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)$

Here is a proof:

$$\dfrac{\dfrac{\dfrac{}{\cdot \vdash \texttt{zero} : \mathbb{N}}\,\texttt{Z} \quad \dfrac{\dfrac{}{x : \mathbb{N} \vdash x : \mathbb{N}}\,\texttt{VAR}}{\cdot \vdash \lambda x : \mathbb{N}.x : \mathbb{N} \to \mathbb{N}}\,\texttt{ABS}}{\cdot \vdash \langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle : \mathbb{N} \times (\mathbb{N} \to \mathbb{N})}\,\texttt{Pair} \quad \dfrac{\dfrac{\dfrac{}{x : \mathbb{N} \vdash x : \mathbb{N}}\,\texttt{VAR}}{x : \mathbb{N} \vdash \texttt{succ}\ x : \mathbb{N}}\,\texttt{S}}{\cdot \vdash \lambda x : \mathbb{N}.\texttt{succ}\ x : \mathbb{N} \to \mathbb{N}}\,\texttt{ABS}}{\cdot \vdash \langle \langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle : \mathbb{N} \times (\mathbb{N} \to \mathbb{N}) \times (\mathbb{N} \to \mathbb{N})}\,\texttt{Pair}}{\cdot \vdash P : \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)}\,\texttt{Pack}$$

Existential Types

Let $P = \begin{array}{rl} \texttt{pack} & \langle \mathbb{N}, \langle \langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle \rangle \\ \texttt{as} & \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha) \end{array}$

Provide a derivation that
$\texttt{unpack}\ \alpha, x\ =\ P\ \texttt{in}\ (\texttt{snd}\ (\texttt{fst}\ x))\ (\texttt{fst}\ (\texttt{fst}\ x))$ has type $\mathbb{N}$

Here is a proof, where $\Gamma = x : \alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)$:

$$\dfrac{\dfrac{\text{[proved above]}}{\cdot \vdash P : \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)} \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash x : \alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)}\,\textsc{VAR}}{\Gamma \vdash \texttt{fst}\ x : \alpha \times (\alpha \to \mathbb{N})}\,\textsc{Fst}}{\Gamma \vdash \texttt{snd}\ (\texttt{fst}\ x) : \alpha \to \mathbb{N}}\,\textsc{Snd} \qquad \Pi}{\Gamma \vdash (\texttt{snd}\ (\texttt{fst}\ x))\ (\texttt{fst}\ (\texttt{fst}\ x)) : \mathbb{N}}\,\textsc{APP}}{\cdot \vdash \texttt{unpack}\ \alpha, x\ =\ P\ \texttt{in}\ (\texttt{snd}\ (\texttt{fst}\ x))\ (\texttt{fst}\ (\texttt{fst}\ x)) : \mathbb{N}}\,\textsc{Unpack}$$

where Π is:

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash x : \alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)}\,\textsc{VAR}}{\Gamma \vdash \texttt{fst}\ x : \alpha \times (\alpha \to \mathbb{N})}\,\textsc{Fst}}{\Gamma \vdash \texttt{fst}\ (\texttt{fst}\ x) : \alpha}\,\textsc{Fst}$$

Existential Types

Let $P = \begin{array}{rl} \texttt{pack} & \langle \mathbb{N}, \langle\langle \texttt{zero}, \lambda x : \mathbb{N}.x \rangle, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle\rangle \\ \texttt{as} & \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha) \end{array}$

Show that $\texttt{unpack}\ \alpha, x\ =\ P\ \texttt{in}\ (\texttt{snd}\ x)\ \texttt{zero}$ is not typable, i.e., we cannot break the abstraction barrier

We can attempt a proof as follows, where $\Gamma = x : \alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)$:

$$\dfrac{\dfrac{\text{[proved above]}}{\cdot \vdash P : \exists \alpha.\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)} \qquad \dfrac{\dfrac{\dfrac{\text{[stuck]}}{\Gamma \vdash x : T \times (\mathbb{N} \to U)}}{\Gamma \vdash \texttt{snd}\ x : \mathbb{N} \to U}\ \texttt{Snd} \qquad \dfrac{}{\Gamma \vdash \texttt{zero} : \mathbb{N}}\ \texttt{Z}}{\Gamma \vdash (\texttt{snd}\ x)\ \texttt{zero} : U}\ \texttt{APP}}{\cdot \vdash \texttt{unpack}\ \alpha, x\ =\ P\ \texttt{in}\ (\texttt{snd}\ x)\ \texttt{zero} : U}\ \texttt{Unpack}$$

We cannot complete this proof because $T \times (\mathbb{N} \to U)$ cannot match $\alpha \times (\alpha \to \mathbb{N}) \times (\alpha \to \alpha)$

Objects

As mentioned in lecture 10:

- objects also provide a way of abstracting over data
- they include some internal state & methods to manipulate that state

Simple objects can be captured using existential types

As opposed to ADTs, invoking the method of an object should return a modified object

Using packages and existential types, we can define the following counter type T and object P with initial value 0:

$$\begin{array}{lcl} T & = & \exists\alpha.\alpha \times ((\alpha \to \mathbb{N}) \times (\alpha \to \alpha)) \\ P & = & \texttt{pack}\,\langle \mathbb{N}, \langle \texttt{zero}, \langle \lambda x : \mathbb{N}.x, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle \rangle \rangle\ \texttt{as}\ T \end{array}$$

This is essentially the ADT presented above.

We simply reorder the elements of the package so that a package is

- a pair of a state
- and a collection of methods

Objects

A counter type T and a counter object P with initial value 0:

$$\begin{array}{lcl} T & = & \exists\alpha.\alpha \times ((\alpha \to \mathbb{N}) \times (\alpha \to \alpha)) \\ P & = & \texttt{pack}\ \langle \mathbb{N}, \langle \texttt{zero}, \langle \lambda x : \mathbb{N}.x, \lambda x : \mathbb{N}.\texttt{succ}\ x \rangle \rangle \rangle\ \texttt{as}\ T \end{array}$$

The increment function on counter objects works as follows:

$$\begin{array}{l} \lambda c : T.\ \texttt{unpack}\ \alpha, x\ =\ c\ \texttt{in} \\ \qquad\qquad \texttt{pack}\ \langle \alpha, \langle (\texttt{snd}\ (\texttt{snd}\ x))\ (\texttt{fst}\ x), \texttt{snd}\ x \rangle \rangle\ \texttt{as}\ T \end{array}$$

it works as follows:

- it unpacks the object,
- applies the increment method to the state,
- and repacks the object with the modified state and the same methods

Conclusion

What did we cover today?

- What are existential types?
- Abstract data types as existential types
- Existential types in System F

Next week

- Monday: consolidation (with me)
- Thursday: consolidation (with Vincent, followed by office hour)
- Friday: guest lecture by Professor Dan Ghica

Reminder

- Assignment 1 released today.